Sprint 3 – Week 1 Progress Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The platform enables:

- Organizations to post structured, well-defined micro-projects.
- Students to browse and discover opportunities.
- Students to apply for projects.
- Organizations to manage and review applications.
- A controlled workflow to manage project participation.
- End-to-end completion of projects through deliverables, review, and feedback.

The system architecture follows a layered full-stack model:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

The platform supports two primary user roles:

- Student
- Organization

1.2 Sprint 1 & Sprint 2 Recap – Workflow Foundation

Sprint 1 delivered the Minimum Viable Marketplace (MVP Foundation), including:

- User authentication and registration (JWT authentication)
- Role-Based Access Control (Student / Organization)
- Student profile management
- Organization profile management
- Project posting
- Basic project browsing and search

Sprint 2 introduced the Application Workflow Layer, enabling:

- Students to apply to projects
- Application tracking and visibility
- Organization-side review and decision-making

User Story	Description
US-1	Registration/Login
US-2	Student Profile
US-3	Organization Profile
US-4	Post Listing
US-5	Browse/Search
US-6	Apply
US-7	Review Applications
US-8	Track Status
US-9	Accept/Reject

These stories satisfied:

- FR-1 through FR-9
- End-to-end workflow from account creation to application decision

Sprint 2 transformed the system into a fully interactive marketplace. Sprint 3 focuses on completing the system lifecycle from accepted application to project completion, feedback, and system governance.

1.3 Sprint 3 & Sprint 4 Roadmap Context

Sprint 3 focuses on completing the core system lifecycle, including:

- Deliverable submission (FR-10)
- Deliverable review (FR-11)
- Project completion (FR-12)
- Feedback system (FR-13)
- Admin moderation (FR-14)
- System logging (FR-15)

Following backlog refinement, a fourth sprint was introduced to extend the platform with additional features:

- Notifications system (US-16)
- Messaging system (US-17)
- Dashboard analytics (US-18)
- Profile enhancements (US-19)

This separation ensures that Sprint 3 remains focused on core functionality, while Sprint 4 introduces enhancements and user experience improvements.

1.4 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14

FR-15: The system shall log system actions.	US-15
FR-16: The system shall notify users of key events including application decisions, deliverable reviews, and project completion.	US-16
FR-17: The system shall allow users to send and receive messages within the context of a project.	US-17
FR-18: The system shall provide users with basic analytics including counts of applications and completed projects.	US-18
FR-19: The system shall allow profiles to display skills, completed projects, and ratings.	US-19

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 3 – Week 1 Overview

2.1 Sprint Goal

The goal of Sprint 3 – Week 1 is:

- To implement the Deliverable Submission and Review Workflow, enabling the transition from accepted applications to completed work.

This sprint establishes the execution phase of the platform, completing the transition from application approval to project fulfillment.

3. Sprint Planning

3.1 Product Backlog Grooming and Reprioritization

At the start of Sprint 3, backlog refinement focused on completing the remaining functional requirements (FR-10 through FR-15) while incorporating newly identified enhancements (FR-16 through FR-19) into a future sprint.

The system at the end of Sprint 2 supported the full application workflow; however, it lacked the ability to execute and complete projects. Therefore, Sprint 3 prioritizes post-acceptance lifecycle functionality.

Remaining Core Requirements

- FR-10 through FR-15

Newly Introduced Requirements

- FR-16 through FR-19 (Sprint 4 enhancements)

Reprioritization Strategy

I used Planning Poker with Fibonacci estimation (3, 5, 8, 13) to reassess story complexity and confirm story point allocations.

Stories were evaluated based on:

- Dependency relationships
- Impact on system completeness
- Risk of integration failure
- Contribution to end-to-end workflow

Priority Decisions

Highest Priority – Core Execution Workflow

- **US-10: Submit Deliverables (5 points)**
- **US-11: Review Deliverables (3 points)**

Rationale:

These stories are essential to transition the system from an approval-based workflow to a functional execution platform.

Without these features:

- Accepted applications cannot produce outcomes
- Projects cannot progress beyond approval
- The system fails to deliver real-world value

These stories introduce the core interaction between students and organizations post-acceptance, making them the most critical for Sprint 3 Week 1.

Medium Priority – Workflow Completion

- **US-12: Mark Project Complete (3 points)**

Rationale:

Project completion depends on successful deliverable submission and review. Implementing this feature before validating submission/review would introduce premature state transitions and increase the risk of inconsistent workflow states.

Lower Priority – Supporting Features

- **US-13: Feedback (5 points)**
- **US-14: Admin Moderation (5 points)**
- **US-15: Logging (8 points)**

Rationale:

- Feedback (US-13) is a post-completion feature, dependent on project closure
- Admin moderation (US-14) is a governance feature, not required for core workflow validation
- Logging (US-15) is a cross-cutting concern, best implemented after all major workflows are stable

Deferring these ensures:

- Reduced complexity in early sprint phases
- Focus on functional correctness

- Better integration stability

Deferred to Sprint 4 – Enhancements

➢ **US-16 to US-19**

Rationale:

These features enhance usability and engagement but do not impact core system functionality.

Deferring them:

- Prevents overloading Sprint 3
- Maintains realistic velocity
- Aligns with Agile incremental delivery

3.2 Groomed Product Backlog

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX

US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern
US-16	Notifications	Medium	5	Sprint 4	User awareness
US-17	Messaging	Medium	8	Sprint 4	Communication
US-18	Analytics	Low	5	Sprint 4	Insights
US-19	Profile Enhancements	Low	3	Sprint 4	UX Improvements

3.3 Sprint Backlog – Week 1 Commitment

Based on my capacity, Week 1 included:

User Story	Description	Story Points
US-10	Submit Deliverables	5
US-11	Review Deliverables	3
Supporting Tasks	Model + API + UI Integration	8

Total Commitment: 16 Story Points

The supporting 8 story points were decomposed into UI, dashboard, and integration tasks (#65, #67, and supporting work), ensuring alignment between user stories and implementation-level issues.

Capacity Planning and Velocity Justification

As a single-developer team, historical velocity across previous sprints averaged:

- Sprint 1: ~16 story points
- Sprint 2: ~14 story points

This establishes a consistent velocity range of 15–18 story points per week.

Sprint 3 Week 1 Complexity Factors

The current sprint includes:

- New database entity (Deliverables)
- Multiple backend endpoints
- Role-based logic (student + organization)
- Frontend UI for both roles
- Integration across workflow stages
- Testing and validation requirements

Commitment Justification

A commitment of 16 story points was selected because it is:

- Within historical velocity range
- Balanced between backend and frontend work
- Realistic given integration complexity
- Sustainable without risking quality

Risk-Controlled Scope

US-12 (completion) and other features were deferred to avoid:

- Parallel workflow dependencies
- Increased debugging complexity
- Integration bottlenecks

This phased approach ensures:

- Stable delivery
- Incremental validation
- Reduced technical risk

3.4 Sprint 3 Backlog Structure

Sprint 3 was divided into three weekly development phases.

Week 1 – Deliverable Workflow

Primary Focus:

- Deliverable submission
- Deliverable review

Completed Issues

Issue	Description	Story Points	User Stories	Status
#64	Submit Deliverable API	5	US-10	Done
#65	Deliverable UI	3	US-10	Done
#66	Review Deliverable API	3	US-11	Done
#67	Deliverable Dashboard	3	US-11	Done
Supporting Work	Integration/Validation	2	US-10/US-11	Done

Week 1 Total Commitment: 16 Story Points

Week 2 – Completion & Platform Features

Primary Focus:

- Completing lifecycle
- Adding user-facing features

Planned Issues

Issue	Description	Story Points
#68	Mark Project Complete API	3
#69	Feedback Submission API	5
#80	Feedback UI Integration	3
#70	Admin Moderation API	5

Week 2 Total Commitment: 16 Story Points

Week 3 — System Quality & Stability

Primary Focus:

- System-wide reliability

- Final validation

Planned Issues

Issue	Description	Story Points
#71	Logging Middleware	8
#81	End-to-End Workflow Testing	5
#82	Bug Fixes & UI Polish	3

Week 3 Total Commitment: 16 Story Points

4. Definition of Done (Sprint 3)

The Definition of Done was refined for Sprint 3 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- Data model is designed and reviewed
- Database schema aligns with requirements
- API contract is clearly defined
- Endpoint request/response structures documented

Implementation

- Database migration successfully applied
- Backend endpoint implemented and functional
- Business logic correctly enforces requirements
- Role-Based Access Control (RBAC) enforced
- Input validation implemented (edge cases handled)
- Frontend components implemented (if applicable)
- Frontend successfully integrated with backend APIs

Testing

- Unit tests written for core logic
- Edge case scenarios covered
- API endpoints tested (success + failure cases)
- Manual functional testing completed

- No failing test cases

Quality & Integration

- Code coverage ≥ 90% for core business logic
- Code reviewed and approved
- Pull request merged into main branch
- No regression introduced to previous features
- System builds and runs without errors

Documentation

- API endpoints documented
- Code comments added where necessary
- User flow documented (if applicable)

Deployment Readiness

- Feature is deployable without breaking system
- Environment variables/configurations updated

5. Acceptance Criteria

5.1 US-10 – Submit Deliverables

Acceptance Criteria:

- Only authenticated students may submit deliverables.
- Student must have an accepted application for the project.
- Application must exist in the system.
- Deliverable must be linked to a valid application.
- Deliverable may include file upload or text-based submission.
- Duplicate submissions are prevented.
- Submission timestamp is recorded.

Returns:

- 201 on success
- 400 if duplicate submission
- 403 if wrong role or unauthorized
- 404 if application not found

5.2 US-11 – Review Deliverables

Acceptance Criteria:

- Only authenticated organization users may review deliverables.
- Deliverable must exist in the system.
- Deliverable must be associated with the organization's project.
- Organization can mark deliverable as "accepted" or "rejected".
- A deliverable cannot be reviewed more than once.
- Status must update correctly in the database.

Returns:

- 200 on success
- 403 if unauthorized role
- 404 if deliverable not found
- 400 if already reviewed

6. Source Code Development Progress

6.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

6.2 Summary of Contributions

Backend Contributions

- Designed and implemented the Deliverable ORM model
- Established relationships between:
  - Applications
  - Projects
  - Users
- Implemented POST /applications/{id}/deliverables
- Implemented GET /deliverables/{id}
- Implemented PUT /deliverables/{id}/review

- Enforced business rules:
    - Only accepted applications can submit deliverables
    - Only organizations can review deliverables
    - Prevent duplicate submissions
    - Prevent multiple reviews
- Integrated RBAC enforcement using dependency injection
- Implemented validation logic for:
    - Missing applications
    - Unauthorized users
    - Invalid states

Frontend Contributions

- Implemented Deliverable Submission UI
    - Input form
    - Submit button
    - Validation messages
- Integrated submission flow with backend API
- Implemented Organization Review Dashboard
    - Display deliverables
    - Accept/Reject buttons
- Added conditional rendering:
    - Student vs Organization views
- Implemented:
    - Loading states
    - Error handling
    - Status updates in UI

Integration Work

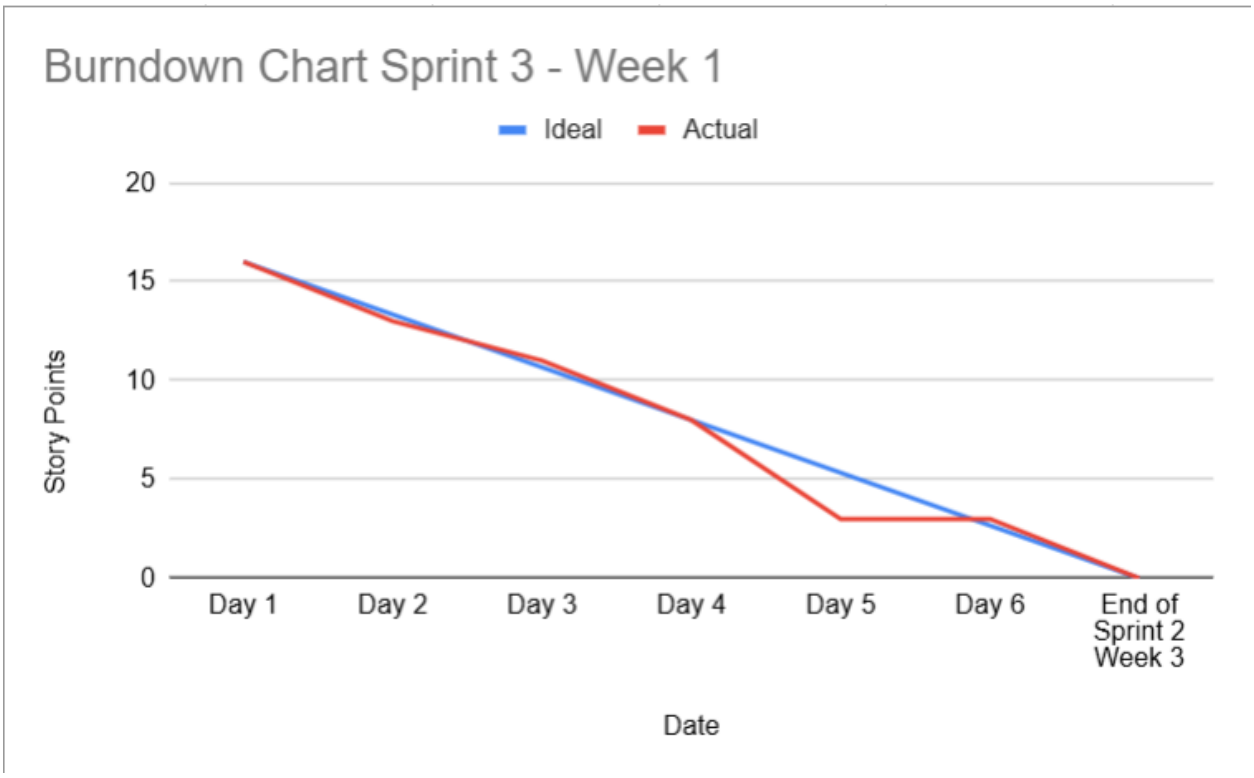
- Connected frontend and backend workflows
- Verified API contract alignment
- Ensured consistent state transitions:
    - Accepted → Submitted → Reviewed
- Validated end-to-end workflow manually

6.3 Important Commits

Commit Message	Related Issue	User Story

feat: submit deliverables endpoint	#64	US-10
feat: review deliverables endpoint	#66	US-11
ui: submission form	#65	US-10
ui: deliverables dashboard	#67	US-11
test: submit deliverables	#64	US-10
test: review deliverables	#66	US-10

6.4 Burndown Chart



Sprint Capacity: 16 Story Points
Sprint Duration: 6 Development Days

Burndown Chart Analysis

The burndown chart reflects steady and controlled progress across six development days, with work completed incrementally as backend, frontend, and integration tasks progressed.

Velocity Alignment:

Sprint 1 average velocity: ~16 points
Sprint 2 average velocity: ~14 points
Sprint 3 Week 1 completed: 16 points

7. Software Testing

7.1 Requirement-to-Test Mapping

Functional Requirement	User Story	Test Case ID
FR-10	US-10	TC-US10-01
FR-10	US-10	TC-US10-02
FR-10	US-10	TC-US10-03
FR-10	US-10	TC-US10-04
FR-10	US-10	TC-US10-05
FR-11	US-11	TC-US11-01
FR-11	US-11	TC-US11-02
FR-11	US-11	TC-US11-03

7.2 User Acceptance Test Specification

All test cases are mapped to FR-10 and FR-11 ensuring full coverage of sprint scope.

TC-US10-01 – Successful Deliverable Submission

Related Requirement: FR-10
User Story: US-10 – Submit Deliverables
Actor: Student

Preconditions:

- Student account exists
- Student is authenticated (valid JWT session)
- Student has an accepted application for a project
- Project exists and is active
- No existing deliverable has been submitted for this application

Steps:

1. Navigate to the login page of the application.

2. Enter valid student credentials (email and password).
3. Click the “Login” button.
4. Verify successful login and redirection to the student dashboard.
5. Navigate to the “My Projects” or “Accepted Applications” section.
6. Select a project with an accepted application status.
7. Click on the project to open the project details page.
8. Locate the “Submit Deliverable” section.
9. Enter valid deliverable content (e.g., link, file reference, or text submission).
10. Click the “Submit Deliverable” button.
11. Confirm submission if a confirmation modal appears.
12. System sends POST request to /applications/{id}/deliverables.

Expected Result:

- HTTP 201 Created is returned
- Deliverable is successfully stored in the database
- Deliverable is linked to the correct application ID
- Submission timestamp is recorded
- Deliverable status is set to “submitted”
- UI updates to reflect successful submission
- Submission button becomes disabled or hidden

Postconditions:

- Deliverable persists in database
- Application state reflects submitted deliverable
- No duplicate deliverable entries exist

TC-US10-02 – Duplicate Submission Prevention

Related Requirement: FR-10

User Story: US-10 – Submit Deliverables

Actor: Student

Preconditions:

- Student is authenticated
- Student has already submitted a deliverable for the selected application

Steps:

1. Log in as a Student using valid credentials.
2. Navigate to the “My Projects” section.

3. Select the same project where a deliverable has already been submitted.
4. Open the project details page.
5. Locate the “Submit Deliverable” section.
6. Attempt to enter new deliverable content.
7. Click the “Submit Deliverable” button again.
8. System sends POST request to /applications/{id}/deliverables.

Expected Result:

- HTTP 400 Bad Request is returned
- Error message displayed: “Deliverable already submitted”
- No new deliverable record is created
- Database still contains exactly one deliverable for the application
- UI prevents further submissions

Postconditions:

- System state remains unchanged
- No duplicate records created

TC-US10-03 – Unauthorized Submission

Related Requirement: FR-10
User Story: US-10 – Submit Deliverables
Actor: Organization

Preconditions:

- Organization account exists
- Organization user is authenticated
- Application exists

Steps:

1. Navigate to login page.
2. Enter valid organization credentials.
3. Click “Login”.
4. Navigate to a project where applications exist.
5. Attempt to access deliverable submission functionality via UI or direct API request.
6. Trigger submission action (button click or POST request).

Expected Result:

- HTTP 403 Forbidden returned
- Error message: “Only students can submit deliverables”
- No deliverable is created in database
- System logs unauthorized attempt (if logging enabled)

Postconditions:

- Database remains unchanged.

TC-US10-04 – Application Not Found

Related Requirement: FR-10
User Story: US-10 – Submit Deliverables
Actor: Student

Preconditions:

- Student is authenticated
- Application ID does not exist

Steps:

1. Log in as Student.
2. Attempt to manually send POST request using invalid application ID.
3. Endpoint: /applications/{invalid_id}/deliverables.

Expected Result:

- HTTP 404 Not Found returned.
- Error message: “Application not found.”
- No deliverable created.

Postconditions:

- No data persisted.

TC-US10-05 – Invalid Deliverable Submission (Empty Input)

Related Requirement: FR-10
User Story: US-10 – Submit Deliverables
Actor: Student

Preconditions:

- Student account exists
- Student is authenticated (valid JWT session)
- Student has an accepted application for a project
- Project exists and is active
- No existing deliverable has been submitted for this application

Steps:

1. Navigate to the login page of the application.
2. Enter valid student credentials (email and password).
3. Click the “Login” button.
4. Verify successful login and redirection to the student dashboard.
5. Navigate to the “My Projects” or “Accepted Applications” section.
6. Select a project with an accepted application status.
7. Click on the project to open the project details page.
8. Locate the “Submit Deliverable” section.
9. Leave the deliverable input field empty (no text, file, or link provided).
10. Click the “Submit Deliverable” button.
11. System sends POST request to /applications/{id}/deliverables.

Expected Result:

- HTTP 400 Bad Request is returned
- Error message displayed: **“Deliverable content cannot be empty”**
- No deliverable is created in the database
- UI displays validation error message near input field
- Submission is blocked and form remains active for correction

Postconditions:

- No data is persisted in the database

- Application remains in “accepted” state
- User is able to retry submission with valid input

TC-US11-01 – Successful Deliverable Review

Related Requirement: FR-11
User Story: US-11 – Review Deliverables
Actor: Organization

Preconditions:

- Organization user is authenticated
- Deliverable exists
- Deliverable belongs to organization’s project
- Deliverable status = “submitted”

Steps:

1. Navigate to login page.
2. Enter valid organization credentials.
3. Click “Login”.
4. Navigate to “Project Dashboard” or “Applications” section.
5. Select project with submitted deliverables.
6. Click on a specific deliverable to open details.
7. Review deliverable content.
8. Click “Accept Deliverable” button.
9. System sends PUT request to /deliverables/{id}/review.

Expected Results:

- HTTP 200 OK returned
- Deliverable status updated to “accepted”
- Database reflects updated status
- UI displays updated status badge

Postconditions:

- Deliverable marked as accepted
- Application moves closer to completion

TC-US11-02 – Reject Deliverable

Related Requirement: FR-11
User Story: US-11 – Review Deliverables
Actor: Organization

Preconditions:

- Deliverable exists
- Status = “submitted”

Steps:

1. Log in as Organization.
2. Navigate to project dashboard.
3. Select deliverable.
4. Click “Reject Deliverable”.
5. Confirm rejection if prompted.
6. System sends PUT request.

Expected Results:

- HTTP 200 OK returned
- Deliverable status updated to “rejected”
- UI reflects rejection

Postconditions:

- Deliverable marked rejected
- Student can resubmit (if allowed by system rules)

TC-US11-03 – Prevent Double Review

Related Requirement: FR-11
User Story: US-11 – Review Deliverables
Actor: Organization

Preconditions:

- Deliverable has already been reviewed (accepted or rejected)

Steps:

1. Log in as Organization.
2. Navigate to reviewed deliverable.
3. Attempt to click “Accept” or “Reject” again.
4. System sends PUT request to review endpoint.

Expected Result:

- HTTP 400 Bad Request returned
- Error message: “Deliverable already reviewed”
- Deliverable status remains unchanged
- UI prevents further interaction

Postconditions:

- No state change occurs

8. Backlog Grooming

During Sprint 3 planning:

- A fourth sprint was introduced
- New functional requirements (FR-16 to FR-19) added
- New user stories (US-16 to US-19) added
- No changes to existing estimates

No changes were made to existing story point estimates during Sprint 3 Week 1; only new Sprint 4 items (US-16–US-19) were introduced during prior backlog refinement.

Rationale

Sprint 3 focuses on completing the core system lifecycle (FR-10 through FR-15).

Sprint 4 focuses on enhancements:

- Notifications
- Messaging
- Analytics
- Profile enhancements

This ensures:

- Stable core system
- Controlled scope
- Improved user experience in later iteration

9. Risk Management

Risk	Mitigation
Duplicate submissions	Validation + DB constraints
Unauthorized access	RBAC enforcement
Workflow inconsistency	State validation
Integration issues	Early integration

10. Sprint 3 Week 1 Outcome

Sprint 3 Week 1 successfully delivered:

- Deliverable submission functionality
- Organization review workflow
- Full system integration across roles

The system now supports the execution phase of the micro-internship lifecycle, moving beyond application approval into project fulfillment. All planned user stories (US-10, US-11) were completed within scope and met Definition of Done.

The sprint goal was fully achieved, and the system is now prepared for completion, feedback, and final enhancements in subsequent weeks.